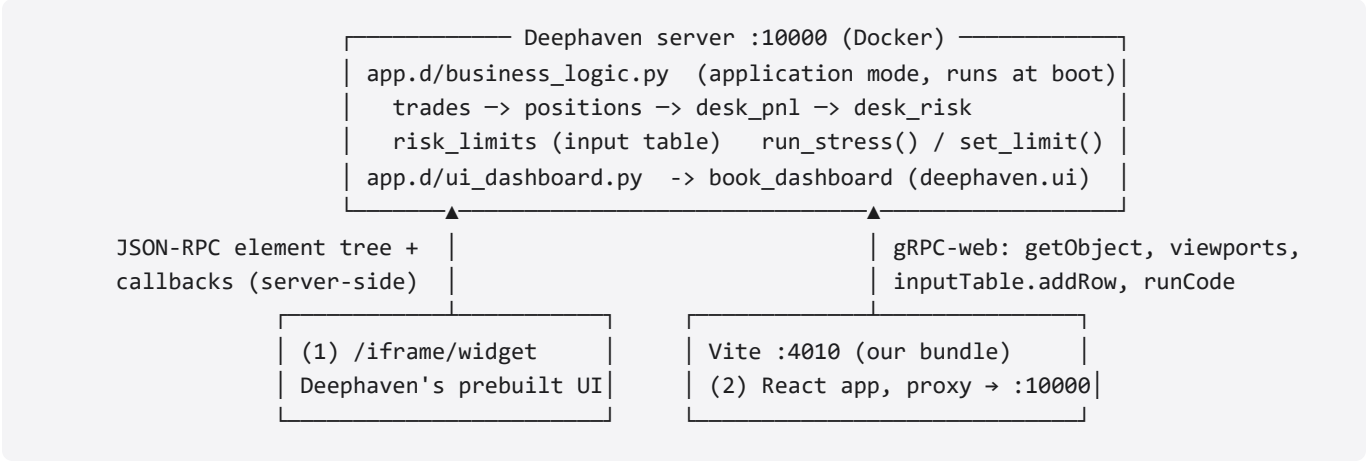
Deephaven app comparison: deephaven.ui vs. web-client-ui

The same small "market-making book monitor" is built twice on top of one Deephaven Community server (v42.6; Docker, or the pip `deephaven-server` run as a plain Python process). Everything we learned is collected in `VERIFIED_FACTS.md`, with how we know each claim: a test in this repo, the installed 42.6 source, or the Deephaven docs.

	(1) deephaven.ui	**(2) React + @deephaven/***
URL	http://localhost:10000/iframe/widget/?name=book_dashboard	http://localhost:4010
Code	`server/app.d/ui_dashboard.py` (~190 lines of Python)	`react-client/src/*` (~350 lines of TS/TSX)
Web server	Deephaven itself (Jetty on :10000)	Vite on :4010, proxying to :10000
Where the UI logic runs	**On the server**, in Python	**In the browser**, in React



Both frontends talk to the **same live tables**. Change a desk limit in one and the other shows the breach within a tick.

Run it

Prereqs: Node 18+, and either Docker Desktop or Java 17+ (`JAVA_HOME`) for the pip server.

```
# Option A: Docker
docker compose up -d --build         # Deephaven on :10000 (first pull ~1.5 GB)
# Option B: no Docker -- the pip server as a plain Python process
py -3.12 -m venv .venv-server; .\.venv-server\Scripts\pip install -r server\requirements-server.txt
.\.venv-server\Scripts\python server\run_server.py
```

```
# Either way, the React app on :4010
cd react-client; npm install; npm run dev
```

The pip server is documented as Linux/macOS/WSL 2 only; `server/run_server.py` makes it run natively on Windows. It needs Java 17+ in `JAVA_HOME`, then applies the other two workarounds itself: JVM DLL folders on `PATH`, and relative web-storage directories. `server/setup_wsl.sh` and `server/start.sh` set it up inside WSL instead.

- (1): http://localhost:10000/iframe/widget/?name=book_dashboard (or the IDE at http://localhost:10000/ide/, under **Panels** → **book_dashboard**)
- (2): http://localhost:4010
- The scripts in `server/app.d` only run at server start, so after editing them restart the server (`docker compose up -d --force-recreate`, or rerun `run_server.py`).
- Auth is **anonymous**, which is fine on localhost only. For a login, replace `-DAuthHandlers=io.deephaven.auth.AnonymousAuthenticationHandler` in `START_OPTS` (`docker-compose.yml`) with `-Dauthentication.psk=<key>`, or run `run_server.py --psk <key>`. `AppBootstrap` in (2) will then show a login prompt.

What each one demonstrates

Feature	(1) deephaven.ui	(2) React + web-client-ui
Live KPI cards	`ui.use_row_data(book_summary)`	`table.setViewport()` + `EVENT_UPDATED` listener (`hooks.ts`)
Live grid	`ui.table(desk_risk, format_=[...])` with server-evaluated conditional formats and databars	`<IrisGrid>` from `@deephaven/iris-grid`. It's the same grid, but deephaven.ui's `format_` (number formats, conditional colors, databars) has no IrisGrid prop, so numbers get the grid's defaults. To match, format the table in Python (`format_columns`, which IrisGrid renders)
Write to server	`set_limit()`, a plain Python call	`table.inputTable()` → `addRow({Desk, Limit})`, with no server code
"Callback" on server	`ui.use_memo(lambda: run_stress(shock, desk), [shock, desk])`	`session.runCode("stress_xxx = run_stress(...)")` → `session.getTable()`
Cross-panel state	`ui.use_state` hoisted to `book_layout` (row click → desk)	React `useState` in `App`
Chart	`dx.line(...)`, one line	Not included. It would need `@deephaven/chart` + plotly wiring
Layout	`ui.dashboard` (drag/dock/tabs for free)	Your own CSS grid, or `@deephaven/dashboard` + golden-layout (heavy)

Takeaways

deephaven.ui is the fastest way to a live, interactive app if your team writes Python. You get no build step, no client code, and Deephaven's grid, plots and docking layout for free. State is per-session automatically, so two users with different sliders don't interfere. The costs: - Every interaction is a server round trip, and rendering spends server CPU per connected user. - You're limited to the component set it ships (React Spectrum + DH widgets). For anything else you have to write a JS plugin. - The page shell (`/iframe/widget`, the IDE) belongs to Deephaven, so branding and routing are limited.

web-client-ui gives full control of the frontend: your own routing, design system, other data sources, and deploying separately from the data server. The costs: - Plumbing. The bootstrap chain is `ApiBootstrap → AppBootstrap → ClientBootstrap → Auth → Connection`, and you manage the lifecycles of tables, models and sessions (`close()` everything). - The packages are monorepo-first, so integration has sharp edges (see below). - "Calling Python" means `runCode` against a **shared global scope**. Each tab here writes to its own `stress_<id>` variable to avoid clobbering others, and those variables are never cleaned up. In production you'd want a proper RPC instead: a custom object-type plugin with a message stream (which is how deephaven.ui itself works), or input tables as a "request" channel.

A reasonable hybrid: put the business logic in Python as tables and functions (as here), use deephaven.ui for internal and analyst tools, and use the React path only for the one or two apps that need a fully custom frontend.

Gotchas hit while building this (and the fixes in the code)

1. **App-mode scope.** The Deephaven docs say application-mode variables are not in the console scope unless declared `global`, so our scripts declare them. Testing later showed this is unnecessary on Community 42.6 Python: the scripts run in the global scope (`globals() is __main__.__dict__`), and undeclared top-level names are published as fields too (`tests/probe_concepts.py`). The declarations are harmless.
2. **`jsapi-types` version.** The web-client-ui 1.3x packages depend on `@deephaven/jsapi-types` 41.x. Installing 42.x (matching the server) creates duplicate, incompatible types. Keep `~41.11`. The *runtime* API always comes from the server (`/jsapi/dh-core.js`), so this only affects types.
3. **`@paciolan/remote-module-loader` interop.** `@deephaven/app-utils` default-imports this CJS module and gets `TypeError: ... is not a function` under Vite. Fix: the alias shim in `react-client/src/shims/` and `vite.config.ts`.
4. **The IrisGrid README is stale.** `<IrisGrid dh={dh} …>` no longer type-checks. `dh` now comes from context, so the call is just `<IrisGrid model={model} />`.
5. **`use_row_data` returns pandas-typed values.** Long columns arrive as floats (`514.0`). Cast with `int()`.
6. **Single origin.** Proxy `/jsapi`, `/js-plugins`, `/arrow.*` and `/io.deephaven.*` (with websockets) from your web server to Deephaven. Then the browser only talks to :4010, and you avoid CORS.

Files

```
VERIFIED_FACTS.md               every claim, with how we know it (test, source or docs)
docker-compose.yml              Deephaven 42.6 (+ Graphviz), app mode, anonymous auth
server/run_server.py            the same server via pip, no Docker (native Windows ok)
server/app.d/book.app           application-mode manifest
```

```
server/app.d/introspection.py    opens the performance logs at boot (full graph ancestry)
server/app.d/business_logic.py   shared mock book: tables + run_stress/set_limit
server/app.d/ui_dashboard.py     approach (1)
react-client/vite.config.ts      approach (2) web server + proxy
react-client/src/main.tsx        bootstrap chain
react-client/src/hooks.ts        useNamedTable / useViewportRows / useIdeSession
react-client/src/LiveGrid.tsx    IrisGrid wrapper
react-client/src/App.tsx         panels
examples/                        the experiment ladder, introspection, graph renderer
introspect/                      dh_introspect.py (in-worker report) + its client
tests/                           probes behind the Verified marks (scopes, fields,
                                 tickets, runtime features, event-driven clock,
                                 embedded thread, subprocess hosting)
```

Docs used: deephaven.ui docs, JS API how-to, application mode, iframes, and the `embed-widget` package in web-client-ui.